



CSS Grid 基础概念

从Flexbox到Grid：掌握二维布局的强大工具

目录

1. 探秘 CSS Grid：一行代码实现真正的响应式布局
2. 目录
3. minmax() 函数：为轨道尺寸设定弹性边界
4. auto-fit vs auto-fill
5. auto-fit vs auto-fill 对比示例
6. 实战示例
7. 实用的响应式模式
8. 总结
- 9.
- 10.

学习目标

理解Grid的本质

掌握Grid与Flexbox的根本区别

建立心智模型

从容器、轨道到网格线的完整概念体系

动手实践

创建第一个Grid布局，理论与实践结合

为进阶打基础

为后续学习复杂Grid特性做好准备

Grid：复杂布局的救星

当Flexbox遇到它的边界时

Flexbox的强项

- 一维布局：沿单轴排列元素
- 空间分配：灵活的对齐和空间控制
- 响应式友好：自动折行和伸缩

遇到的挑战

- 二维矩阵布局：仪表盘、图库
- 精确定位需求：圣杯布局、复杂表格
- 行列同时控制：需要更精细的控制

一维 vs. 二维

理解这个区别是掌握Grid的关键

Flexbox：一维布局

```
.container {  
  display: flex;  
  flex-direction: row; /* 或 column */  
}
```

沿单轴排列

- 主轴方向：水平或垂直
- 交叉轴：垂直于主轴
- 换行：自动到下一行/列



类比：书架

像把书一字排开放在书架上 书太多就自动换到下一层

Grid：二维布局

```
.container {  
  display: grid;  
  grid-template-columns: 200px 100px 200px;  
  grid-template-rows: 100px 100px;  
}
```

行列双向控制

- 同时定义行和列
- 精确的网格单元格
- 元素可跨越多个格子



类比：书柜

预先定制好多行多列的书柜 明确指定某本书放在第几行第几列

Grid的核心要素

理解这三个概念是掌握Grid的关键



Grid容器 (Grid Container)

```
.container {  
  display: grid; /* 声明为Grid容器 */  
}
```

一旦声明，内部的直接子元素自动成为**Grid项**



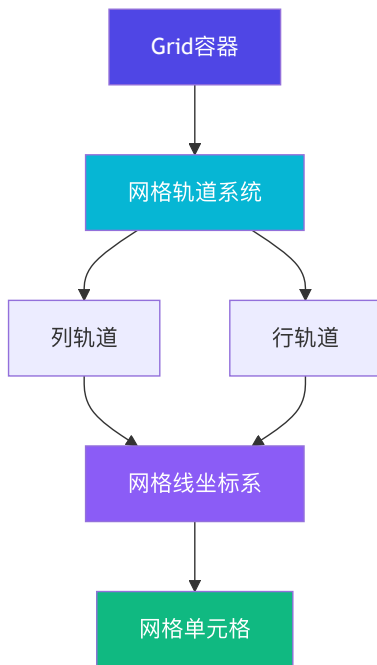
网格轨道 (Grid Tracks)

- 行轨道：水平方向的"走廊"
- 列轨道：垂直方向的"走廊"
- 内容就在这些轨道里排列



网格线 (Grid Lines)

- 分割轨道的**坐标系**
- n 列网格 = $n+1$ 条垂直线
- m 行网格 = $m+1$ 条水平线



定义网格蓝图

`grid-template-columns` & `grid-template-rows`

grid-template-columns

定义网格的**列**轨道。

```
.container {  
  /* 定义三列：第一列200px，第二列占1份剩余空间，第三列100px */  
  grid-template-columns: 200px 1fr 100px;  
}
```

grid-template-rows

定义网格的**行**轨道。

```
.container {  
  /* 定义两行：第一行高100px，第二行高度由内容决定 */  
  grid-template-rows: 100px auto;  
}
```

- 语法与 `grid-template-columns` 几乎完全相同。

gap

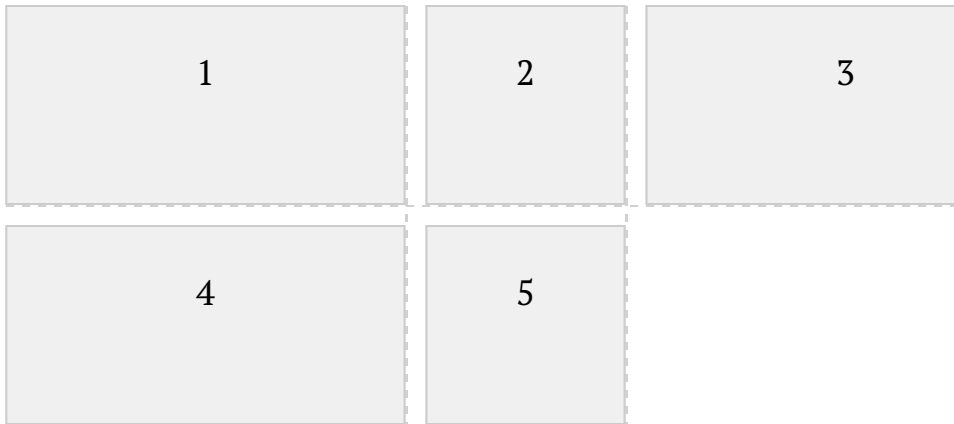
定义网格轨道之间的**间距**。

```
.container {  
  /* 行和列的间距都是 10px */  
  gap: 10px;  
  
  /* 行间距20px，列间距10px（上/下，左/右） */  
  gap: 20px 10px;  
}
```

动手实践：第一个Grid

让我们创建一个3列2行的网格布局

```
.grid-container {  
  display: grid;  
  /* 定义三列，宽度分别为 200px, 100px, 200px */  
  grid-template-columns: 200px 100px 200px;  
  /* 定义两行，高度均为 100px */  
  grid-template-rows: 100px 100px;  
  /* 增加轨道间距 */  
  gap: 10px;  
}  
  
.item {  
  background-color: #f0f0f0;  
  border: 1px solid #ccc;  
  padding: 20px;  
  text-align: center;  
}
```



建立正确的心智模型

从思维方式的转变开始

"内容驱动"到"结构驱动"

- **Flexbox**: 内容决定布局
- **Grid**: 先规划蓝图, 再填充内容

从"一维思考"到"二维思考"

- 同时考虑行和列
- 网格就是你的画布
- 元素在画布上精确定位

实际应用场景

- **仪表盘布局**: 多个组件精确定位
- **响应式网格**: 不同屏幕尺寸的适配
- **复杂表单**: 标签和输入框的对齐

学习建议

循序渐进 先掌握基础概念, 再学习高级特性

多做练习 理论结合实践, 动手才能真正理解

建立直觉 通过大量练习, 培养对Grid的直觉感知

总结与展望

今天的收获

我们建立了Grid布局的核心心智模型

Grid = 二维布局工具

先画蓝图，再填内容

三大要素：容器 + 轨道 + 网格线

这是Grid世界的基础框架

实践出真知

动手创建第一个Grid布局

下一步学习方向

- **Grid项的精确定位：** grid-area、grid-column等
- **响应式Grid布局：** fr单位、minmax()函数
- **Grid模板区域：** grid-template-areas的强大功能

感谢观看！



恭喜你迈出了Grid学习的第一步

记住：Grid不是更复杂的Flexbox
它是专为二维布局而生的强大工具